Rapport de projet

Déploiement de serveurs minecraft avec ansible

Logan Quaghebeur

Etudiant BTS SIO – Option SISR

Ecole AFTEC

Table des matières

Introduction projet 3

I. Contexte 3

II. Expression des besoins 4

III. Approche du projet 5

1. Justification du choix de l'hyperviseur 5

2. Ressources minimales requises 6

3. Démarche de mise en œuvre 7

Mise en œuvre 8

I. Configuration serveur Minecraft 8

1. Création d'un conteneur 8

2. Installation serveur Minecraft 10

II. Création des playbooks ansible 13

1. Ansible & Playbook 13

2. Configuration Ansible-Maitre et Proxmox 16

3. Playbook de configuration serveur 20

4. Playbook de provisionnement conteneur 23

5. Sécurisation 28

Introduction projet

I. Contexte

Dans le cadre de ma deuxième année de BTS Services Informatiques aux Organisations (option SISR – Solutions d'Infrastructure, Systèmes et Réseaux), il m'a été demandé de réaliser un projet technique permettant de mettre en pratique les compétences acquises en administration système, virtualisation et automatisation.

J'ai choisi de travailler sur un projet interne portant sur la mise en place d'une infrastructure d'automatisation à l'aide d'Ansible, hébergée sur un environnement virtualisé Proxmox.

L'objectif principal de ce projet est de concevoir une solution capable de déployer automatiquement plusieurs serveurs Minecraft sur des conteneurs, via un serveur Ansible maître.

Cette automatisation permettra d'assurer un déploiement rapide, homogène et reproductible de plusieurs instances de serveurs Minecraft, tout en facilitant leur mise à jour et leur maintenance.

II. Expression des besoins

L'objectif général est de disposer d'une infrastructure capable de déployer et de gérer automatiquement plusieurs serveurs Minecraft à partir d'un seul point de contrôle. Les besoins identifiés sont les suivants :

- Mettre en place un environnement virtualisé sur Proxmox.
- Créer un serveur Ansible maître dans un conteneur dédié.
- Configurer plusieurs conteneur cibles pouvant héberger des serveurs Minecraft.
- Créer des playbooks Ansible permettant :
  - L'installation automatique des dépendances nécessaires (Java, paquets système, etc.)
  - le déploiement et la configuration d'un serveur Minecraft
  - Le provisionnement d'un ou plusieurs conteneurs LXC
  - la mise à jour simultanée de plusieurs serveurs,
  - et éventuellement la sauvegarde ou la suppression automatisée d'instances.
- Documenter la procédure et les résultats afin d'assurer la reproductibilité du projet.

III. Approche du projet

1. Justification du choix de l'hyperviseur

Parmi les hyperviseurs gratuits disponibles sur le marché, plusieurs solutions pouvaient être envisagées, telles que VMware ESXi Free, VirtualBox ou encore Proxmox VE.

Après comparaison, le choix s'est porté sur Proxmox VE pour plusieurs raisons. En effet Proxmox est basé sur Debian, c'est un système d'exploitation que je maîtrise et que j'apprécie pour sa stabilité et sa flexibilité. Il permet aussi de gérer à la fois des machines virtuelles et des conteneurs LXC. Enfin ce système intègre nativement des fonctionnalités de sauvegarde, de gestion réseau et de stockage, ce qui simplifie grandement l'administration.

Voici un tableau récapitulatif des caractéristiques de chaque hyperviseur :

Critère	VMware ESXi	VirtualBox	Proxmox VE ★ Choix retenu
💰 Coût	Payant Licence commerciale onéreuse. Version gratuite supprimée en 2024.	Gratuit Open source (GPL). Aucun coût de licence.	Gratuit Open source (AGPL v3). Support entreprise optionnel.
📦 Gestion LXC	Non supporté Uniquement VM (VMX). Pas de conteneurs LXC natifs.	Non supporté Uniquement VM. Pas de gestion de conteneurs.	Natif & complet LXC intégré nativement. Idéal pour conteneurs légers.
🖥️ Interface Web	vSphere Client Interface web complète. Nécessite vCenter pour fonctions avancées.	Absente Interface graphique locale uniquement. Pas d'interface web native.	Intégrée & moderne Interface web complète sans dépendance. Accessible dès l'installation.
👥 Communauté	Professionnelle Grande communauté entreprise. Documentation VMware officielle.	Active Forums Oracle & communauté open source. Surtout orienté desktop/test.	Large & active Forums officiels très actifs. Forte présence homelab & pro.
🛠️ Support	Commercial uniquement Support officiel payant. SLA garanti selon contrat.	Communautaire Pas de support officiel dédié. Forums et documentation Oracle.	Communautaire + Pro Support communautaire gratuit. Abonnement entreprise disponible.
⚡ Performances	Excellentes Hyperviseur de type 1. Optimisé pour workloads critiques.	Moyennes Hyperviseur de type 2. Overhead dû à l'OS hôte.	Très bonnes Hyperviseur de type 1 (KVM). LXC quasi-natif sans overhead.

2. Ressources minimales requises

Pour garantir un fonctionnement fluide de l'infrastructure, certaines ressources matérielles sont nécessaires à l'installation et à l'exploitation de Proxmox ainsi que des machines hébergées.

Serveur hôte (Proxmox VE) :

- Processeur : 4 cœurs minimum
- Mémoire vive : 8 Go minimum (16 Go recommandés pour héberger plusieurs VM ou conteneurs)
- Stockage : 100 Go minimum sur disque SSD ou HDD rapide
- Accès réseau stable pour la gestion web et SSH

Conteneur Ansible maître :

- Système d'exploitation : Debian ou Ubuntu Server
- 2 à 4 Go de RAM
- 1 à 2 vCPU
- 10 Go d'espace disque
- Logiciels : Ansible, Python, OpenSSH

Machines cibles (serveurs Minecraft) :

- 2 à 4 Go de RAM par instance
- 1 vCPU minimum
- 10 à 20 Go d'espace disque selon la taille du monde
- Dépendances : OpenJDK (Java), screen, wget

Ces prérequis garantissent une bonne répartition des ressources entre les différents éléments du projet et assurent la stabilité du système, même lors du déploiement simultané de plusieurs serveurs Minecraft.

3. Démarche de mise en œuvre

La démarche suivie pour ce projet repose sur plusieurs étapes :

1) **Configuration d'un serveur Minecraft sans ansible**

Configuration d'un conteneur « test » pour y installer un premier serveur Minecraft. Pour cette première étape nous n'utiliserons pas ansible et le déploiement automatique. Nous installerons un serveur manuellement afin de comprendre et de valider les composants et services nécessaire au déploiement d'un serveur.

Une fois l'installation valider nous pourrons passer au déploiement automatique avec Ansible.

2) **Création et mise en place d'un conteneur ansible maître**

Création du conteneur Ansible maître. C'est à partir de ce conteneur que nous pourrons déployer automatiquement un serveur Minecraft sur un conteneur cible à l'aide d'un Playbook ansible.

3) **Développement des playbooks Ansible**

Les playbooks Ansible nous permettront d'automatiser l'installation des dépendances nécessaires pour héberger un serveur Minecraft, de déployer et configurer les serveurs Minecraft et de mettre à jour automatiquement les serveurs.

Nous pourrons aussi créer un playbook permettant de provisionner un ou des conteneurs sur le Proxmox.

4) **Déploiement et tests**

Enfin nous pourrons provisionner des conteneurs et déployer les serveurs sur ces derniers depuis le conteneur maître et réaliser les tests pour valider le bon fonctionnement

Cette approche structurée permet d'aborder le projet de manière méthodique, tout en démontrant les compétences en virtualisation, automatisation et gestion d'infrastructure attendues dans le cadre du BTS SIO option SISR.

Mise en œuvre

I. Configuration serveur Minecraft

Dans un premier temps, nous allons déployer un serveur Minecraft manuellement sans Ansible. Cette étape nous permettra d'analyser les dépendances nécessaires au bon fonctionnement d'un serveur Minecraft.

1. Création d'un conteneur

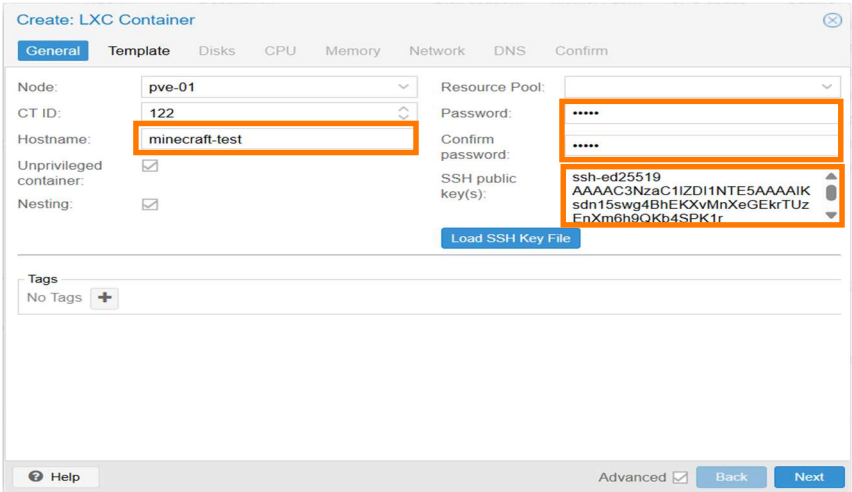
Sur notre hyperviseur Proxmox, nous allons créer un premier Conteneur.

Pour ce faire nous allons nous rendre sur l'environnement web de notre Proxmox :

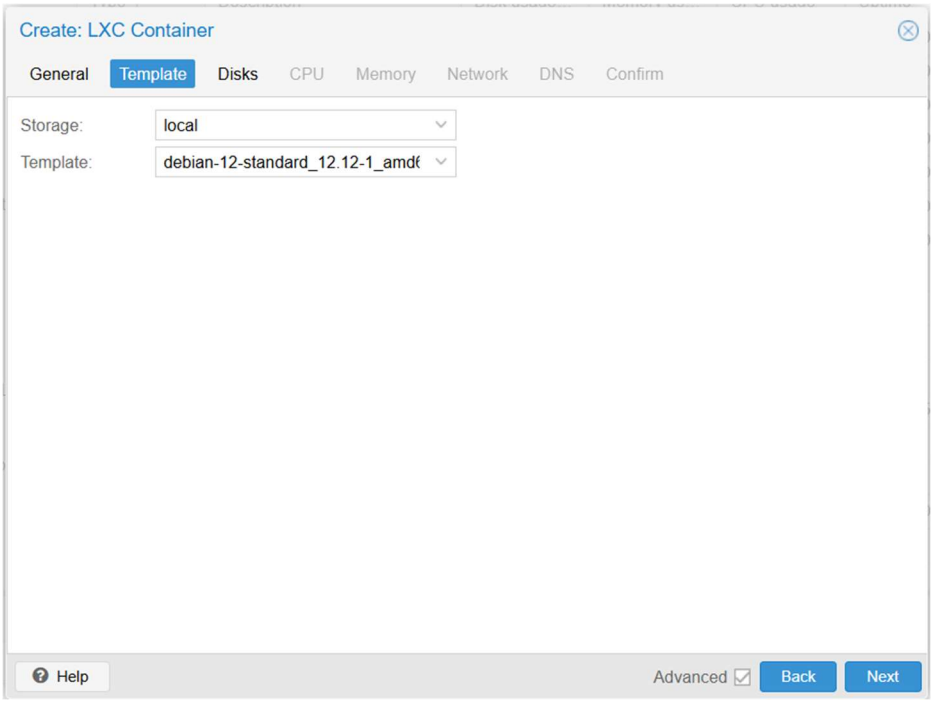
- Cliquer sur Create CT :

Nous allons ensuite configurer ce premier conteneur.

- Nous allons choisir le hostname, ici « minecraf-test » et choisir un mot de passe « root ». J'en profite pour rentrer ma clé SSH publique pour pouvoir piloter le conteneur depuis la console de mon ordinateur :



- Choisir le système d'exploitation. Pour ma part je vais utiliser une image Linux Debian



12.

- Je vais ensuite continuer chaque étape de la configuration avec les éléments suivant :

Taille du disque dur : 20 GiB

CPU : 2 Cores

Memory : 2048 MiB

IP : 10.1.0.25 /24

Gateway : 10.1.0.254

Une fois la configuration terminée je peux démarrer le conteneur.

2. Installation serveur Minecraft

Dans un premier temps on va préparer le serveur Minecraft.

1. Mettre à jour les paquets avec leur dernière version :

```
sudo apt update
sudo apt full-upgrade
```

2. On va vérifier si tous les paquets nécessaires sont installés :

```
sudo apt install screen nano wget git
```

3. On va ensuite installer le paquet java. On va aller chercher la dernière version du paquet sur le site oracle :

```
sudo wget https://download.oracle.com/java/21/latest/jdk-21_linux-x64_bin.deb
sudo dpkg -i jdk-21_linux-x64_bin.deb
```

4. Afin de sécuriser notre système et éviter les vulnérabilités, on va créer un utilisateur « minecraft » qui exécutera les actions du serveur :

```
sudo adduser minecraft --disabled-password
```

Puis on bascule sur le nouvel utilisateur :

```
sudo su - minecraft
```

5. Enfin on va créer un dossier nommé « server » :

```
mkdir ~/server && cd ~/server
```

6. Dans ce dossier on va télécharger le logiciel Minecraft server. Copier le lien sur le site https://minecraft.net/download/server puis utiliser wget :

```
wget lien_de_telechargement
```

7. Il faut ensuite accepter la licence du logiciel. Pour cela on va taper la commande suivante :

```
echo "eula=true" > eula.txt
```

Un fichier est alors créé à la racine de votre serveur, contenant la ligne eula=true. Cela indique au logiciel que vous acceptez les conditions d'utilisations de Minecraft.

Le serveur est prêt à être lancé !

Pour lancer le serveur Minecraft on va utiliser le paquet « Screen » installer précédemment. Cela permet d'exécuter plusieurs sessions du terminal shell, cela permet par exemple de lancer plusieurs serveur Minecraft simultanément.

Tout d'abord nous allons créer un nouveau shell nommé minecraft1 :

```
screen -S minecraft1
```

Puis dans ce shell, nous allons lancer le serveur Minecraft avec la commande suivante. (Utiliser -ls pour vérifier le nom du fichier)

```
java -Xmx1024M -Xms1024M -jar server.jar nogui
```

Pour information :

- **Xms1024m** : Configure le serveur pour qu'il démarre avec 1024Mo de RAM.
- **Xmx1024m** : Permet au serveur d'utiliser un maximum de 1024Mo de RAM
- **Jar** : Spécifie le fichier jar du serveur à exécuter.
- **Nogui** : cela indique au serveur de ne pas lancer d'interface graphique.

Une fois le serveur opérationnel, vous obtenez le résultat suivant :

```
[14:52:58] [Server thread/INFO]: Done (41.530s)! For help, type "help"
```

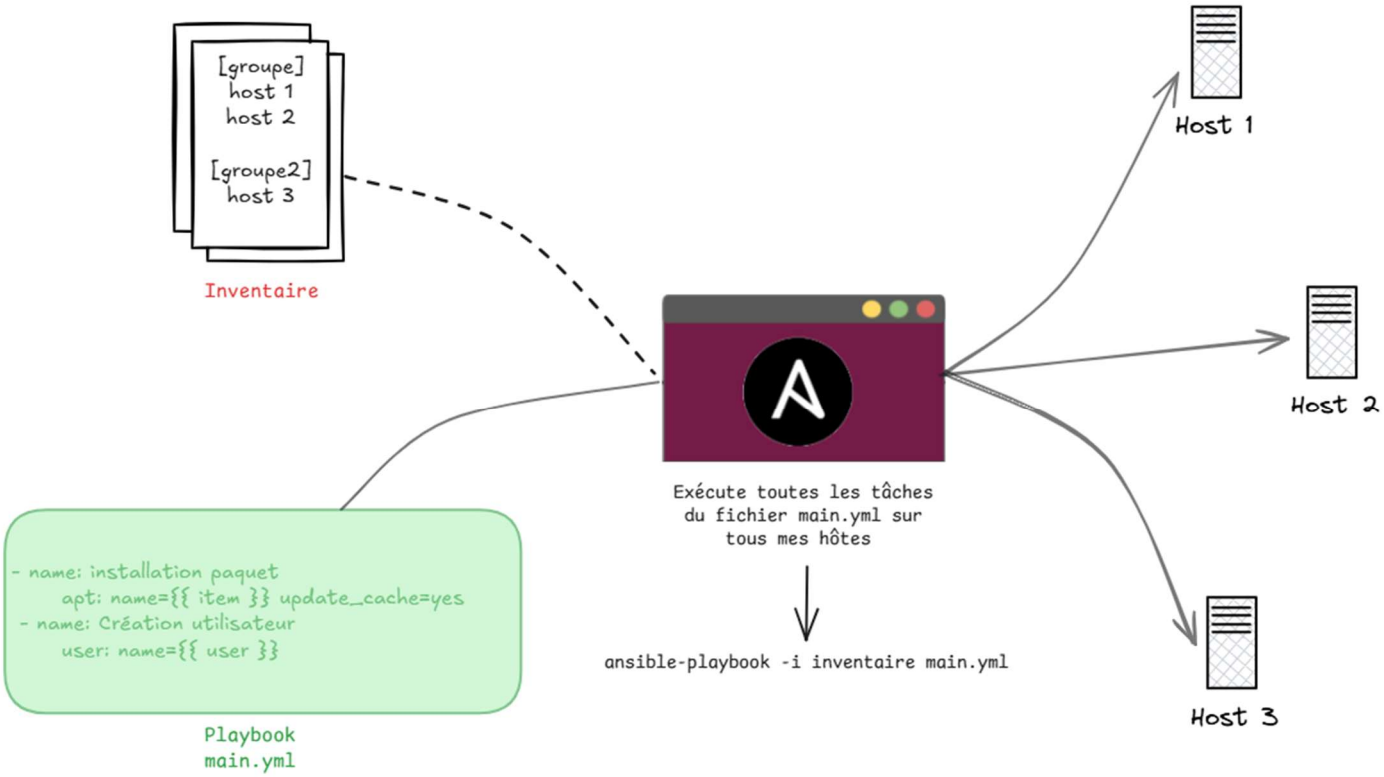
Pour arrêter le serveur, entrer la commande « stop ».

II. Création des playbooks ansible

1. Ansible & Playbook

Maintenant que nous avons configuré un serveur Minecraft manuellement, nous allons pouvoir nous intéresser au fonctionnement d'ansible et des playbooks.

Le schéma ci-dessous illustre, pour exemple, le fonctionnement général d'Ansible :

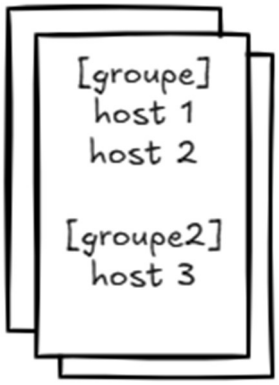


Ansible fonctionne selon trois éléments principaux :

1. L'inventaire

L'inventaire est un fichier qui répertorie les machines sur lesquelles Ansible doit intervenir. On peut y définir des groupes de serveurs afin d'appliquer des configurations différentes selon les besoins.

Dans le schéma, l'inventaire contient par exemple :



Cet inventaire permet à Ansible de savoir où exécuter les tâches du playbook, en l'occurrence host 1, host 2 et host 3.

2. Le playbook (main.yml)

Le playbook est un fichier écrit en YAML décrivant les tâches à réaliser sur les hôtes. Chaque tâche correspond à une action automatisée : installer un paquet, créer un utilisateur, déployer un conteneur, copier un fichier de configuration, etc.

Dans l'exemple illustré dans le schéma :

```
- name: installation paquet
    apt: name={{ item }} update_cache=yes
- name: Création utilisateur
    user: name={{ user }}
```

Un playbook peut contenir autant de tâches que nécessaire pour automatiser complètement le déploiement d'un service.

3. L'exécution du playbook

Une fois l'inventaire et le playbook créé et fonctionnel, l'administrateur exécute simplement la commande :

ansible-playbook -i inventaire main.yml

Ansible se charge alors de :

1. Lire l'inventaire pour identifier les hôtes cibles.
2. Se connecter automatiquement à chacun d'eux (via SSH).
3. Exécuter toutes les tâches du playbook dans l'ordre.

Comme dans le schéma, Ansible envoie les instructions vers chaque machine (Host 1, Host 2, Host 3), ce qui permet de configurer plusieurs serveurs en une seule commande.

2. Configuration Ansible-Maitre et Proxmox

- **Ansible-Maitre**

Tous les playbooks seront lancés depuis un conteneur que l'on va appeler « Ansible-maitre ». Pour cela on va reprendre exactement la procédure du chapitre « Mise en œuvre » I-1.

Une fois le conteneur créé on va installer ansible avec les commandes suivantes :

```
apt update
apt upgrade
apt install ansible
```

Comme vu précédemment, Ansible communique principalement en SSH. Il est donc important de générer la clé SSH public du conteneur :

```
ssh-keygen
```

Puis on va créer le fichier « hosts » qui sera notre inventaire avec les machines à configurer :

```
nano hosts
```

Avec cette configuration le conteneur « Ansible-maitre » est prêt à configurer d'autres conteneurs. Toute fois pour pouvoir provisionner des conteneurs sur le Proxmox ce n'est pas suffisant.

Pour pouvoir provisionner un conteneur sur le Proxmox depuis mon conteneur Ansible-maitre nous allons devoir installer le module **community.general.proxmox** d'Ansible.

Pour ce faire on va installer les dépendances python :

```
apt update
apt install -y python3-pip python3-requests
apt install python3-proxmoxer
```

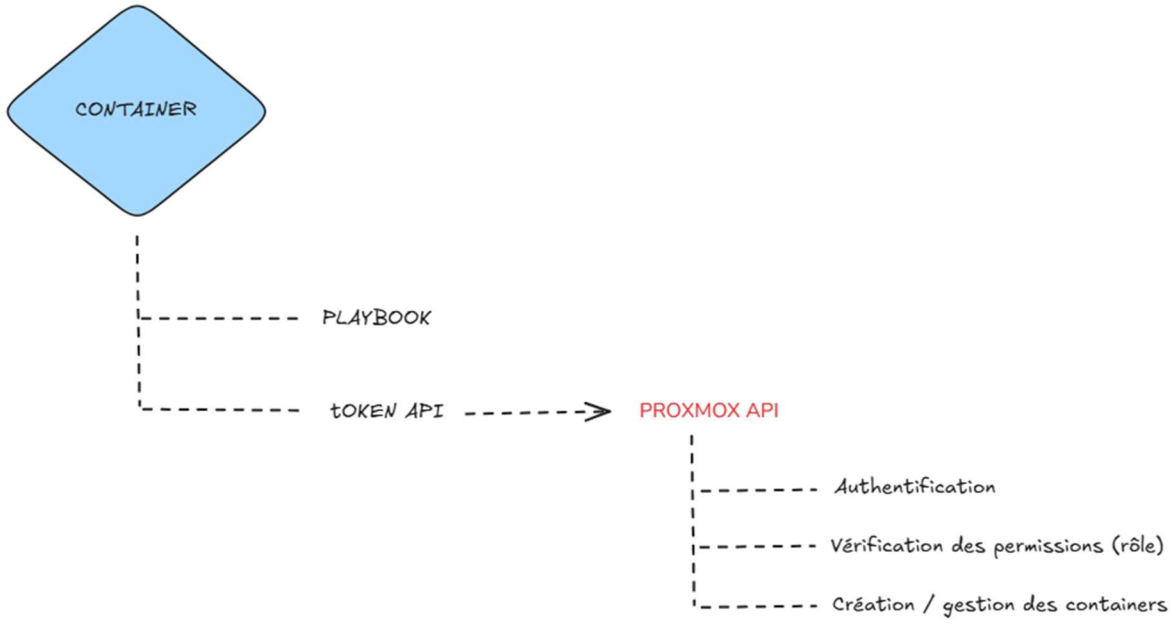
Le conteneur Ansible-maître est désormais opérationnel.

- **Proxmox**

Pour qu'un conteneur Ansible puisse provisionner des conteneurs sur un Proxmox, il faut configurer un système d'authentification sécurisé basé sur :

- Un utilisateur dédié
- Un rôle avec les permissions nécessaires
- Un token API pour l'authentification sans mot de passe

Voici un schéma d'architecture :



Etape 1 : Créer un utilisateur dédié

Sur le Proxmox on va créer un utilisateur « ansible » avec les commandes suivantes :

```
pveum user add ansible@pve
pveum passwd ansible@pve
```

Etape 2 : Créer un rôle personnalisé avec les permissions nécessaires

Voici les permissions requises pour provisionner des conteneurs :

- **VM.Allocate** : Créer/supprimer des VMs/conteneurs
- **VM.Config.*** : Configurer les conteneurs (CPU, RAM, disque, réseau, etc.)
- **VM.Console** : Accès console (optionnel)
- **VM.PowerMgmt** : Démarrer/arrêter/redémarrer
- **Datastore.AllocateSpace** : Allouer de l'espace disque
- **Datastore.Audit** : Lister les datastores
- **Pool.Allocate** : Si vous utilisez des pools de ressources

Pour créer un rôle avec ces permissions nous allons faire la commande suivante :

```
pveum role add AnsibleProvisioner -privs
"Datastore.Allocate,Datastore.AllocateSpace,Datastore.Audit,VM.Allocate,VM.Audit,
VM.Config.CDROM,VM.Config.CPU,VM.Config.Disk,VM.Config.HWType,VM.Config.Memory,VM
.Config.Network,VM.Config.Options,VM.PowerMgmt"
```

On vérifier que le rôle a bien été créé :

```
pveum role list
```

Etape 3 : Assigner le rôle à l'utilisateur

Il faut maintenant associer le rôle **AnsibleProvisioner** à l'utilisateur **ansible@pve** sur les ressources appropriées. Voici les commandes pour le faire :

```
pveum acl modify / -user ansible@pve -role AnsibleProvisioner
```

Puis on vérifier les permissions :

```
pveum acl list
```

Etape 4 : Créer un token API

Les tokens permettent une authentification sans exposer le mot de passe.

Pour ce faire :

```
pveum user token add ansible@pve provisioning --privsep 0
```

Privsep 0 permet au token d'hériter des permissions de l'utilisateur. Il faut bien garder la valeur du token qui s'affiche après la commande ! On devra l'utiliser pour la suite…

3. Playbook de configuration serveur

On va d'abord créer un playbook qui permettra de configurer un conteneur pour qu'il puisse héberger un serveur Minecraft.

Dans un premier temps on va définir, le nom descriptif du playbook, sur quels hôtes il va s'exécuter, l'utilisateur avec lequel Ansible va se connecter en SSH sur les machines distantes ainsi que les variables que l'on va utiliser dans ce playbook :

```
- name: Installation serveur minecraft
  hosts: minecraft
  remote_user: root
  vars:
     user: minecraft
```

Ensuite on va établir les différentes tâches exécuter par le playbook. La première sera l'installation des paquets nécessaires, c'est-à-dire screen, nano, wget et git :

```
  tasks:
    - name: installation paquet
      apt: name={{ item }} update_cache=yes
      with_items:
        - screen
        - nano
        - wget
        - git
```

On va ensuite réaliser une à une les tâches que nous avons réalisé précédemment lors de l'installation d'un serveur Minecraft manuellement :

- L'installation d'openjdk

```
- name: installation openjdk
  ansible.builtin.apt:
    deb: /home/java
  become: yes
```

- La création d'un utilisateur (ici on utilise la variable)

```
- name: Création utilisateur
  user: name={{ user }}
```

- Ajout de la clé SSH

```
- name: ajout de la clef ssh
  ansible.builtin.authorized_key: user={{ user }} key="{{ lookup('file',
'~/.ssh/id_rsa.pub') }}"
```

- Créer le dossier serveur

```
- name: création dossier serveur
  ansible.builtin.file:
    path: /home/minecraft/server
    state: directory
    mode: '0755'
    owner: minecraft
    group: minecraft
```

- Installation du serveur Minecraft

```
    - name: installation minecraft server
      ansible.builtin.get_url:
        url: https://piston-
data.mojang.com/v1/objects/95495a7f485eedd84ce928cef5e223b757d2f764/server.jar
        dest: /home/minecraft/server
```

- Création du fichier « eula=true »

```
    - name: création fichier eula
      copy:
        dest: /home/minecraft/server/eula.txt
        content: 'eula=true'
```

Le playbook est maintenant opérationnel. Il permet dorénavant de configurer un serveur Minecraft automatiquement sur un conteneur.

4. Playbook de provisionnement conteneur

Maintenant on va créer un playbook qui a pour objectif d'automatiser la création, la configuration et le démarrage de plusieurs conteneurs LXC sur un hyperviseur Proxmox.

Il permet également d'enregistrer automatiquement les conteneurs dans un fichier d'inventaire Ansible afin de pouvoir les administrer par la suite.

Comme précédemment on vient préciser le nom descriptif du playbook, le groupe d'inventaire, le type de connexion et les variables :

```
- name: Creation d'un container lxc
  hosts: localhost
  connection: local
  vars:
    container_count: 5
    base_vmid: 301
    base_hostname: "testct"
    base_ip: "10.10.10.201"
    template: "local:vztmpl/debian-12-standard_12.12-1_amd64.tar.zst"
    ssh_public_key: "{{ lookup('file', '~/.ssh/id_rsa.pub') }}"
    gateway: "10.10.10.1"
    netmask: "24"
    cores: 2
    memory: 1024
```

Le bloc « vars » permet de centraliser les paramètres du playbook, pour les réutiliser par la suite. Il est alors facile de réutiliser et modifier le playbook sans toucher aux tâches.

Ici on a :

- conteneur_count : nombre de conteneurs à créer
- base_vmid : identifiant de départ des conteneurs Proxmox
- base_hostname : préfixe des noms des conteneurs
- base_ip : adresse IP de départ
- template : template LXC utilisé (Debian 12)
- ssh_public_key : clé publique SSH pour l'accès sécurisé
- gateway et netmask : paramètres réseau
- cores, memory : ressources allouées à chaque conteneur

Ensuite on passe aux tâches, la première étant la création du conteneur LXC :

```
  tasks:
    - name: Cr  ation d'un container
      community.general.proxmox:
        api_host: "px.lebaron.sh"
        api_user: "logan@pam"
        api_token_id: "ansible-token"
        api_token_secret: "0241ba01-df63-445d-a406-9357f2d2c80c"
        node: "px"
        vmid: "{{ base_vmid + index }}"
        hostname: "{{ base_hostname }}{{ '%02d' | format(index + 1) }}"
        ostemplate: "{{ template }}"
        storage: "local"
        cores: "{{ cores }}"
        memory: "{{ memory }}"
        swap: "{{ memory }}"
        netif:
          net0: "name=eth0,bridge=vmbr1,ip={{ base_ip.rsplit('.', 1)[0] }}.{{
base_ip.rsplit('.', 1)[1]|int + index }}/{{ netmask }},gw={{ gateway }}"
        password: "root!"
        unprivileged: false
        state: present
        pubkey: "{{ ssh_public_key }}"
      loop: "{{ range(0, container_count) | list }}"
      loop_control:
        index_var: index
```

Cette tâche utilise le module **community.general.proxmox**, qui permet de piloter Proxmox via son API.

- Pour chaque itération de la boucle :
- Un conteneur LXC est créé
- Le vmid est incrémenté automatiquement
- Le nom d'hôte est généré dynamiquement (testct01, testct02, etc.)
- Une IP statique unique est attribuée
- Les ressources CPU et mémoire sont définies
- Une clé SSH est injectée pour l'accès sécurisé

La boucle :

```
loop: "{{ range(0, container_count) | list }}"
```

Elle permet de créer plusieurs conteneurs sans dupliquer le code.

Une fois que les conteneurs sont créés plusieurs actions sont nécessaires pour les rendre opérationnels et exploitables par les outils d'administrations. La partie suivante va permettre de démarrer les conteneurs, de s'assurer que leur réseau est fonctionnel puis de les intégrer automatiquement dans **l'inventaire ansible**.

On démarre automatiquement les conteneurs :

```
- name: Démarrer le container
  community.general.proxmox:
    api_host: "px.lebaron.sh"
    api_user: "logan@pam"
    api_token_id: "ansible-token"
    api_token_secret: "..."
    vmid: "{{ base_vmid + index }}"
    state: started
  loop: "{{ range(0, container_count) | list }}"
```

Cette tâche utilise de nouveau le module **community. general.proxmox** pour piloter Proxmox via son API.

Le paramètre **state : started** indique que chaque conteneur doit être démarré. Grâce à la boucle, tous les conteneurs précédemment crées sont démarrés automatiquement sans intervention manuelle.

On laisse ensuite un temps après le démarrage :

```
- name: Attendre que le container obtienne une IP
  pause:
    seconds: 10
```

Cela permet de laisser le temps au système de terminer sa phase de démarrage, d'initialiser les interfaces réseaux et d'appliquer les paramètres IP.

Une fois les conteneurs opérationnels, on vient récupérer dynamiquement les adresses IP :

```
- name: Récupérer l'IP du container
  shell: |
    ssh root@px.lebaron.sh "pct exec {{ base_vmid + index }} -- ip -4 addr show eth0 | grep -
oP '(?<=inet\s)\d+(\.\d+){3}'"
  register: container_ip
  retries: 5
  delay: 5
  until: container_ip.rc == 0
```

Le principe est le suivant :

- Une connexion SSH est établie vers le serveur Proxmox
- Une commande **pct exec** permet d'éxecuter une commande directement à l'intérieure du conteneur.
- La commande affiche l'adresse IPv4
- Un filtre permet d'extraire l'adresse IP qui est stocké dans la variable **conteneur_ip** grâce au paramètre **register**
- On réalise jusqu'à 5 tentatives avec un délai de 5 secondes pour éviter les erreurs liées au démarrage réseau plus lent.

Ensuite on vient créer le groupe de conteneurs lxc_conteneurs dans le fichier d'inventaire hosts :

```
- name: Créer/vider le groupe lxc_containers dans le fichier hosts
  lineinfile:
    path: ./hosts
    line: "[lxc_containers]"
    create: yes
    state: present
```

Ce groupe servira a regrouper tous les conteneurs LXC afin de pouvoir les cibler facilement dans de futurs playbooks notamment les playbooks de configuration.

Pour finir on ajoute tous les conteneurs de la boucle dans le fichier d'inventaire :

```
- name: Ajouter les containers au fichier hosts (avec IP statique)
  lineinfile:
    path: ./hosts
    line: "{{ base_hostname }}{{ '%02d' | format(index + 1) }} ansible_host={{ ... }}"
    insertafter: "^\[lxc_containers\]"
  loop: "{{ range(0, container_count) | list }}"
```

Chaque conteneur est automatiquement ajouté dans le fichier d'inventaire avec son nom d'hôte et son adresse IP statique.

L'option **insertafter** permet, comme son nom l'indique, d'insérer les nouvelles lignes juste après le groupe **[lxc_conteneurs].** Grâce à cette automatisation les conteneurs deviennent immédiatement accessible pour Ansible.

Ce playbook permet d'automatiser entièrement le déploiement de conteneurs LXC sur Proxmox. Il gère la création, la configuration des ressources et du réseau, le démarrage des conteneurs, la récupération de leurs adresses IP et la mise à jour automatique de l'inventaire.

5. Sécurisation

Actuellement les informations sensibles sont écrites en clair dans les playbooks. Ainsi ils sont facilement accessibles surtout en cas de versionning sur github ce qui est une grande vulnérabilité de sécurité.

Pour répondre à ce besoin de sécurisation, **Ansible Vault** a été introduit. Il s'agit d'un outil intégré à Ansible qui permet de chiffrer et de déchiffrer des fichiers sensibles directement dans les playbooks.

Ansible Vault fait partie intégrante du package Ansible, il n'y a donc pas besoin de l'installer. On peut vérifier que l'outil est bien disponible avec la commande suivante :

```
ansible-vault --version
```

On peut dès maintenant utiliser Ansible Vault !

Ansible Vault permet de d'intégrer des fichiers chiffrés directement dans les playbooks Ansible. Pour ce faire on va créer un fichier contenant nos données sensibles.

Dans notre cas il s'agira d'api_token_secret :

```
touch secret.yml
nano secret.yml
```

Ces commandes permettent de créer un fichier « secret.yml » puis de l'éditer. On introduit dans ce fichier le token secret.

```
token_secret: "0241ba01-df63-445d-a406-9357f2d2c80c"
```

On va maintenant chiffrer ce fichier avec la commande **ansible-vault encrypt** :

```
ansible-vault encrypt secret.yml
```

Une fois la commande lancée, vous serez invité à rentrer un **mot de passe** qui sera nécessaire pour déchiffrer le fichier.

On peut ensuite référencer le fichier directement dans le playbook Ansible en utilisant le module **vars_files.** Ce fichier sera alors déchiffré à l'exécution !

Pour utiliser ce fichier dans un playbook, voici ce qu'il faut faire :

```
  vars_files:
    - secret.yml
```

Puis :

```
api_token_secret: {{ token_secret }}
```

Dorénavant lorsque l'on exécutera le playbook, Ansible demandera le mot de passe Vault

Pour déchiffrer secret.yml.

Exécution d'un playbook avec des fichiers Vault

Lorsque vous lancez un playbook qui contient des fichiers chiffrés avec Ansible Vault, Ansible doit pouvoir les déchiffrer au moment de l'exécution. Pour cela, il est nécessaire de fournir le mot de passe utilisé lors du chiffrement. Plusieurs méthodes permettent de le faire.

1. Saisie Manuelle du mot de passe
2. Utilisation d'un fichier contenant le mot de passe

Afin d'automatiser la solution, on va utiliser la deuxième option. Nous allons créer un fichier avec le mot de passe du fichier :

```
echo "MotDePasse" > password_secret.txt
```

Puis on lance le playbook avec la commande suivante :

```
ansible-playbook playbook.yml --vault-password-file password_secret.txt
```

Attention il faut protéger ce fichier, ce dernier doit être accessible uniquement à ceux qui en ont besoin.

On va autoriser la lecture et l'écriture au **propriétaire uniquement** :

```
chmod 600 password_secret.txt
```

On va aussi ajouter le fichier password_secret.txt dans le .gitignore cela permet de s'assurer que ce fichier ne sera pas versionné dans git lors d'un commit.

À ajouter :

- Nettoyage du fichier known_hosts
- Permission du fichie Authorized_keys

À ajouter :

- Faire une partie test d'un serveur Minecraft

A FAIRE :

Petit partie test rapide avec les screens du résultat ansible

Puis un screen dans Minecraft une fois connecté !